

CENG114 – Computer Programming II

Lab Guide #7: Abstract Classes, Interfaces, and String Operations

Instructor: Yusuf Evren AYKAÇ

Week: 10 (Spring 2025–2026)

Assistants: Çağın ÖZKAYA, Hatice UYSAL, Yusuf Ekrem KEÇİLİOĞLU

Lab: 7

Learning Objectives

By the end of this lab, you will be able to:

- Design a class hierarchy that mixes **abstract classes** (for shared behaviour) with **interfaces** (for pluggable capabilities).
- Apply the **Template Method** pattern in an abstract base class.
- Chain different processors polymorphically through a common supertype.
- Manipulate text using only `charAt`, `length` and the `StringBuffer` API.
- Compare the performance of immutable `String` concatenation against mutable `StringBuffer` builders.
- Implement the Knuth–Morris–Pratt string-search algorithm without using `indexOf` or `regex`.

Scenario — NLP Preprocessing Pipeline

Before any NLP model (sentiment analysis, machine translation, text classification) can understand raw text, the text must be cleaned and normalised. A typical user-generated review looks like this:

```
" OMG!! This movie was S000 good,,, LOVED it!!! "
```

A **preprocessing pipeline** transforms that mess into something a model can actually consume:

```
["omg", "this", "movie", "was", "sooo", "good", "loved", "it"]
```

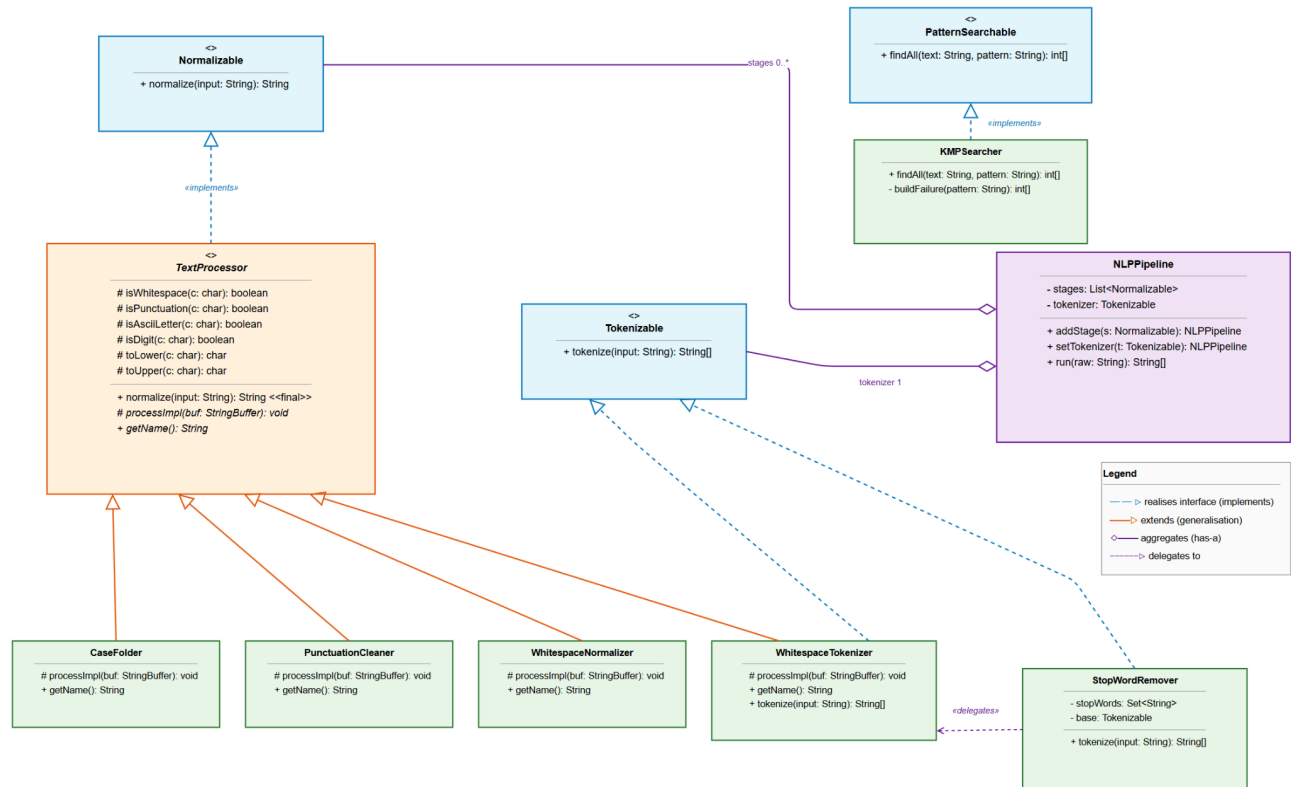
You have been hired as a junior engineer at **NovaText AI** to build exactly this kind of pipeline as a reusable Java module. Each preprocessing step is a *separate responsibility* — lowercase conversion, punctuation removal, whitespace normalisation, tokenisation, stop-word removal. New team members should be able to add new steps without modifying existing code.

This is the ideal setting to practise two complementary Java tools: an **abstract class** that carries the shared text-handling machinery, and **interfaces** that describe what each component can do. Your finished pipeline should allow fluent chaining:

```
String[] tokens = new NLPPipeline()
    .addStage(new CaseFolder())
    .addStage(new PunctuationCleaner())
    .addStage(new WhitespaceNormalizer())
    .setTokenizer(new WhitespaceTokenizer())
    .run(" OMG!! This Movie was S000 good!! ");
```

Class Hierarchy

Study the UML class diagram below before you start coding. The diagram shows how the three interfaces, the TextProcessor abstract base class, and all concrete processors fit together — along with the NLPPipeline that orchestrates them.



Reading the diagram

- `Normalizable`, `Tokenizable` and `PatternSearchable` are interfaces — open (dashed) arrowheads point from an implementer to the interface it realises.
- `TextProcessor` is an **abstract class**. It implements `Normalizable` and provides character-classification helpers plus a template method `normalize`. Concrete processors extend it (closed arrowheads).
- `WhitespaceTokenizer` both extends `TextProcessor` and implements `Tokenizable` — it carries two arrows, one of each kind.
- `NLPPipeline` composes a list of `Normalizable` objects and one `Tokenizable` — the diamond-ended arrows denote aggregation.

Constraints — Read Carefully

⚠ These rules are the whole point of the lab. The TAs will check your source code before marking functionality.

From the `String` class, you may use **ONLY**:

- `charAt(int index)`
- `length()`

From StringBuffer you may use anything:

append, insert, delete, deleteCharAt, replace, reverse, charAt, length, setCharAt, toString, setLength, substring — all allowed.

Forbidden

- String.split, String.replace, String.replaceAll, String.substring, String.indexOf, String.lastIndexOf, String.contains, String.startsWith, String.endsWith, String.trim, String.strip, String.toLowerCase, String.toUpperCase, String.equals, String.equalsIgnoreCase, String.concat, String.matches, String.chars
- java.util.regex.* — any regex use is forbidden.
- Character.toLowerCase / Character.toUpperCase / Character.isLetter / Character.isDigit — classify characters manually using ASCII arithmetic.
- The + operator on String inside loops — use StringBuffer.append instead. Exception: the performance benchmark class (that is the whole point).

Allowed imports

java.util.ArrayList, java.util.List, java.util.HashSet, java.util.Set, java.util.Arrays (for Arrays.asList only). Nothing else.

Tasks**Task 1 — Define the interfaces**

Create the two required interfaces exactly as shown:

```
public interface Normalizable {
    String normalize(String input);
}

public interface Tokenizable {
    String[] tokenize(String input);
}
```

Keep them minimal — one abstract method each, no default methods.

Task 2 — Build the abstract TextProcessor

Create TextProcessor as an **abstract class** that implements Normalizable and provides:

Protected character-classification helpers

All manual — do not call any Character.* method.

```
protected boolean isWhitespace(char c) // space, tab, \n, \r
protected boolean isPunctuation(char c) // . , ! ? ; : " ' ( ) [ ] { } - _ / \
protected boolean isAsciiLetter(char c) // A-Z, a-z
protected boolean isDigit(char c) // 0-9
protected char toLower(char c) // 'A'-'Z' -> 'a'-'z'
protected char toUpper(char c) // 'a'-'z' -> 'A'-'Z'
```



Hint: use ASCII arithmetic. 'A' is 65, 'a' is 97, so the gap is exactly 32.

A final template method

```
public final String normalize(String input) {
    if (input == null) return "";
    StringBuffer buf = new StringBuffer();
    for (int i = 0; i < input.length(); i++) {
        buf.append(input.charAt(i));
    }
    processImpl(buf);           // subclass hook
    return buf.toString();
}
```

Abstract hooks

```
protected abstract void processImpl(StringBuffer buf);
public abstract String getName();
```

⚠ Subclasses must mutate buf in place using StringBuffer methods — do not create a new StringBuffer inside processImpl.

Task 3 — CaseFolder

Extend `TextProcessor` and override `processImpl` to convert every character in the buffer to lowercase using your `toLowerCase` helper plus `StringBuffer.setCharAt`.

Test input: "HeLLo WORLD 123!" → expected result: "hello world 123!".

Task 4 — PunctuationCleaner

Extend `TextProcessor` and remove every character for which `isPunctuation(c)` is true.

Walk the buffer **backwards** and call `deleteCharAt(i)`. Add a one-line comment that explains why walking forwards while deleting breaks the indices.

Test input: "Hello, world!! (really?)" → expected result: "Hello world really".

Task 5 — WhitespaceNormalizer

Collapse any run of whitespace characters into a **single space** and trim leading/trailing whitespace. Manual, single $O(n)$ pass — no trim, no regex.

Algorithm:

1. Build a new `StringBuffer` out.
2. Keep a boolean `prevWasSpace = true` so leading whitespace is skipped.
3. For each char: if whitespace and `!prevWasSpace`, append one ' '; if not whitespace, append the char; update the flag.
4. If out ends with ' ', `deleteCharAt(out.length() - 1)`.
5. Replace the contents of buf with out: `buf.setLength(0); buf.append(out);`

Test input: " Hello\t\t world \n\n " → expected result: "Hello world".

Task 6 — WhitespaceTokenizer

This class **extends** `TextProcessor` (for the inherited helpers) *and* implements `Tokenizable`. `processImpl` is a no-op — tokenising does not mutate the text.

`tokenize(String)` splits on runs of whitespace without using `split`:

1. Walk the input with `charAt`. Append non-whitespace chars into a per-token `StringBuffer`; on whitespace (or end of string), if the token buffer is non-empty, add `token.toString()` to an `ArrayList<String>` and reset the buffer.
2. Return `list.toArray(new String[0])`.

Test input: "hello world foo\tbar" → expected result: ["hello", "world", "foo", "bar"].

Task 7 — NLPPipeline

The pipeline stores a `List<Normalizable>` and a `Tokenizable`, applies the normalisers in order, then tokenises the result.

```
public class NLPPipeline {
    private List<Normalizable> stages = new ArrayList<>();
    private Tokenizable tokenizer;

    public NLPPipeline addStage(Normalizable stage) { /* append, return this */ }
    public NLPPipeline setTokenizer(Tokenizable t) { /* set, return this */ }

    public String[] run(String rawText) {
        // 1) apply each Normalizable in order
        // 2) tokenize the result
    }
}
```

The fluent `addStage` / `setTokenizer` signatures return `this` so that the `Main` class can chain calls.

Task 8 — Driver Main

Write a `Main` class whose `main` method:

1. Builds the three-stage pipeline shown in the Scenario section.
2. Runs it on at least **three** different dirty inputs — one with tabs, one with multi-line whitespace, one with mixed case / digits / punctuation.
3. Prints the raw input and the resulting token array for each case.

Bonus Tasks (Optional)

Bonus 1 — StopWordRemover

Create a new class `StopWordRemover` implements `Tokenizable` that takes a base `Tokenizable` and a `String[]` of stop words in its constructor. `tokenize` first delegates to the base tokenizer, then filters out tokens found in the stop-word set.

Because `String.equals` is forbidden, write a private helper `boolean charEquals(String a, String b)` that walks both strings with `charAt` and returns `true` iff they have the same length and the same characters.

Default English stop words for testing: {"a", "an", "the", "is", "was", "were", "of", "to", "and", "it"}.

Bonus 2 — KMP Pattern Search

Define a third interface:

```
public interface PatternSearchable {
    int[] findAll(String text, String pattern); // returns all start indices
}
```

Implement the **Knuth–Morris–Pratt** algorithm in a class `KMPSearcher` implements `PatternSearchable`:

- Build the failure (prefix) function using only `charAt`.
- No calls to `indexOf` anywhere in the file.
- Return an `int[]` of match positions — empty array if no matches, never null.

Test: find "ana" in "banana and ananas" → expected result: [1, 3, 11, 13].

Expected Output

Your Main driver must produce output in exactly this format:

```
=== Raw input ===
"  OMG!! This Movie was SOOO good,,,  LOVED it!!!  "

[CaseFolder]          -> "  omg!! this movie was sooo good,,,  loved it!!!  "
[PunctuationCleaner]  -> "  omg this movie was sooo good  loved it  "
[WhitespaceNormalizer] -> "omg this movie was sooo good loved it"
[WhitespaceTokenizer] -> [omg, this, movie, was, sooo, good, loved, it]
```

For the bonus KMP task, your program must also print:

```
=== KMP demo ===
text    = "banana and ananas"
pattern = "ana"
matches = [1, 3, 11, 13]
```

Hints & Common Pitfalls

1. **Deleting while iterating:** if you walk a `StringBuffer` forwards with `deleteCharAt`, indices shift. Walk backwards (`for (int i = buf.length()-1; i >= 0; i--)`) or build a new `StringBuffer`.
2. **Comparing characters:** `buf.charAt(i) == ' '` is fine. Do *not* try `String.valueOf(buf.charAt(i)).equals(" ")`.
3. **Template method discipline:** `normalize` in the abstract class is `final`. Do *not* override it in subclasses.
4. **Immutability trap:** `String s = "abc"; s.replace(...);` does not modify `s` — `String` is immutable. Use `StringBuffer` whenever you need to mutate text.
5. **Null safety:** your `normalize(null)` should return `""`, not throw.
6. **Instance-of vs extends:** `TextProcessor` is a class — you extend it. `Tokenizable` is an interface — you implement it. `WhitespaceTokenizer` does both.